

Rayce Ramsay (1009734888)
Terry Tian (1007663468)
Grant Hamblin (1009860447)
Hardik Shah (1009453690)

CSC490 Assignment A6 - Testing Your Project - 10% of Final Grade

Due: Nov 28th, 11pm with a Github PR + Quercus Submission

The goal of this assignment is to showcase your testing and debugging approach for your project.

1 Part One - Showing Code Coverage (25 marks)

Using `pytest-cov` (<https://github.com/pytest-dev/pytest-cov>) or a similar library, show the code coverage of your project. Create a github workflow to generate your test coverage automatically on your readme and PRs. You can reference the example in `pytest-cov` or other open source projects.

Backend testing utilizes `pytest` and `pytest-cov`, with associated test files located in `src/application/backend/tests`. Code coverage and test results are automatically commented on every Pull Request (PR) that modifies backend files, managed by the GitHub workflow defined in `backend_test.yaml`. Additionally, the total backend test coverage is displayed via a badge updated on the main README when new backend code is pushed to the main branch.

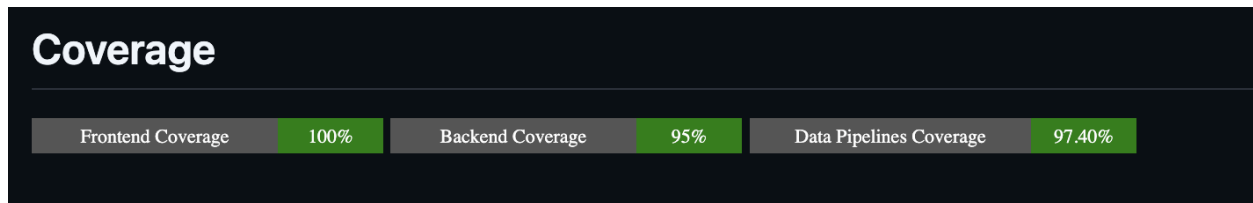
Frontend testing uses Jest for testing and coverage, with files located at `src/application/frontend/src/__tests__`. Similarly, coverage and test results are commented on every PR modifying frontend files, controlled by the GitHub workflow in `frontend_coverage.yaml`. Just like the backend, the total frontend test coverage is displayed via a badge updated on the main README when new frontend code is pushed to the main branch.

Data pipeline testing is implemented using `pytest` and `pytest-cov`, with test suites located in both `src/data_pipelines/tests` and `src/data_extraction/tests`. These tests validate the full ETL workflow, including dataset extraction utilities, S3 synchronization helpers, raw-data processing, and PostgreSQL loading logic. A dedicated GitHub Actions workflow (`data_pipeline_test.yaml`) automatically runs the test suite and generates coverage reports for every Pull Request that modifies data pipeline files. The workflow posts pass/fail status and coverage results for both of these folders.

2 Part Two: Test Coverage Breadth (25 marks)

Let's make sure our code is well tested. You'll receive 0.25 marks for each 1% of code coverage up to 25 marks based on part one.

See code coverage statistics on the A6 branch readme. The readme shows badges with line coverage as follows:



Below are test results from the frontend, backend, data processing, and data extraction pipelines, respectively:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	99.94	98.52	100	99.94	
app	100	100	100	100	
layout.tsx	100	100	100	100	
not-found.tsx	100	100	100	100	
page.tsx	100	100	100	100	
app/(dashboard)	100	100	100	100	
layout.tsx	100	100	100	100	
app/(dashboard)/dashboard	100	100	100	100	
layout.tsx	100	100	100	100	
page.tsx	100	100	100	100	
app/(dashboard)/dashboard/predict	100	100	100	100	
page.tsx	100	100	100	100	
app/(login)/login	100	100	100	100	
page.tsx	100	100	100	100	
app/(login)/signup	100	100	100	100	
page.tsx	100	100	100	100	
components/features/auth	100	100	100	100	
auth-required.tsx	100	100	100	100	
user-context.tsx	100	100	100	100	
components/forms	100	100	100	100	
login-form.tsx	100	100	100	100	
prediction-form.tsx	100	100	100	100	
signup-form.tsx	100	100	100	100	
components/layout	100	100	100	100	
dashboard-header.tsx	100	100	100	100	
header.tsx	100	100	100	100	
components/ui	99.72	92	100	99.72	
alert.tsx	99.17	80	100	99.17	75
button.tsx	100	100	100	100	
card.tsx	100	100	100	100	
input.tsx	100	100	100	100	
label.tsx	100	100	100	100	
loading-screen.tsx	100	100	100	100	
select.tsx	100	100	100	100	
textarea.tsx	100	100	100	100	
hooks	100	100	100	100	
useUser.ts	100	100	100	100	
utils	100	100	100	100	
api.ts	100	100	100	100	
general.ts	100	100	100	100	
testing.ts	100	100	100	100	

Name	Stmts	Miss	Cover	Missing
app/__init__.py	0	0	100%	
app/config/__init__.py	0	0	100%	
app/config/security.py	20	5	75%	10-14
app/db/__init__.py	0	0	100%	
app/dependencies.py	34	0	100%	
app/main.py	29	4	86%	14-20
app/repositories/__init__.py	0	0	100%	
app/repositories/ddi_repository.py	25	0	100%	
app/repositories/role_repository.py	11	0	100%	
app/repositories/user_repository.py	24	0	100%	
app/routers/__init__.py	0	0	100%	
app/routers/auth.py	21	0	100%	
app/routers/predictions.py	28	4	86%	55, 65-67
app/routers/users.py	19	0	100%	
app/services/__init__.py	0	0	100%	
app/services/auth_service.py	21	0	100%	
app/services/prediction_service.py	63	0	100%	
app/services/user_service.py	4	0	100%	
app/utils/__init__.py	0	0	100%	
app/utils/exception_handlers.py	18	2	89%	21, 23
app/utils/helpers.py	32	3	91%	39-41
TOTAL	349	18	95%	

===== tests coverage =====				
coverage: platform darwin, python 3.13.5-final-0				
Name	Stmts	Miss	Cover	Missing
load_to_postgres.py	76	1	99%	228
pipeline.py	308	27	91%	354, 498-507, 510-512, 556-558, 571-576, 633, 645, 714-715
sync_raw_from_s3.py	30	1	97%	48
tests/test_load_to_postgres.py	164	1	99%	93
tests/test_pipeline.py	290	2	99%	7, 563
tests/test_sync_raw_from_s3.py	85	5	94%	9, 100-101, 108, 158
TOTAL	953	37	96%	
Coverage XML written to file coverage.xml				

===== tests coverage =====				
coverage: platform darwin, python 3.13.5-final-0				
Name	Stmts	Miss	Cover	Missing
__init__.py	0	0	100%	
fetch_datasets.py	62	0	100%	
s3_utils.py	30	0	100%	
tests/test_fetch_datasets.py	102	0	100%	
tests/test_s3_utils.py	61	2	97%	9, 40
tests/test_utilities.py	202	1	99%	14
utilities.py	131	0	100%	
TOTAL	588	3	99%	
Coverage XML written to file coverage.xml				

3 Part Three: Test Coverage Depth (20 marks)

Choose your most complex module in your codebase. Create at least 10 unit tests for this module that test edge cases, failure modes and important use cases. Link the tests and edge cases, and explain why you chose these tests in comments of that module. We'll try to find additional edge cases that exist to cause an error! Make sure to write positive tests and negative tests (ones that throw errors or gracefully handle incorrect inputs)

Our most complex module is the `/predict` endpoint, which handles the full RAG workflow: retrieving contextual information, constructing the model prompt, invoking AWS Bedrock, and returning a structured response. Although this workflow is conceptually treated as a single logical unit, its processes span multiple components and are captured in separate pytest classes.

To ensure clear separation of concerns, we organize our backend code into three primary directories:

- **Routers** (`app/routers/`) – Defines FastAPI route handlers responsible for HTTP request/response handling and orchestrating calls to the service layer.
- **Services** (`app/services/`) – Implements core business logic, including data transformation, authentication (e.g., JWT creation and validation), and communication with external systems such as AWS Bedrock.
- **Repositories** (`app/repositories/`) – Acts as the data access layer, issuing SQLAlchemy queries to perform CRUD operations and more complex database interactions while abstracting database details from higher layers.

Our testing strategy mirrors this architecture by isolating dependencies between layers using mocks. Each component is tested through unit tests designed for its respective functions. Consequently, the most complex test suite is the prediction service tests (`tests/services/test_prediction_service.py`). This suite presents the highest likelihood of failure due to its reliance on multiple inputs—user-provided request data, database results, and LLM outputs—which must all be mocked and validated carefully.

See codebase for test functions and comments.

4 Part Four: Two Memorable Bugs (40 marks)

Record a video walkthrough of two of the most memorable bugs you encountered during your project. For each bug, include:

- What the bug was and how you discovered it
- The debugging process you used to identify the root cause
- How you fixed the bug
- What you learned from the experience
- The test cases you created to prevent this bug from recurring

Link your video in your a6.pdf

Link to Bug 1 Vid:

<https://drive.google.com/file/d/1Oho85lgUxK1e6YPXAkk7eEnwOXEUL1m8/view?usp=sharing>

Link to Bug 2 Vid:

<https://drive.google.com/file/d/149NWs1r-xF3Me77BitoSvZI7bhhaRWwx/view?t=5>

5 Submission Instructions

Submit a pull request to the course GitHub repo with your assignment in a folder named a6 with your a6.pdf on a branch called a6, include your team members names on the first page with student IDs. Also upload your assignment to quercus.